

Enci/ VENTAS

Modelo de Datos Firestore

ANEXO D

Proyecto Enci / Enci Ventas

Aplicación de Automatización de Fuerza de Ventas

Equipo de Desarrollo: Grupo PLM - Universidad Tecnológica Metropolitana

Líder: Joshua Flores

Integrantes: Falon Berrios, Carolaine Palacios y Eliseo Poblete

Versión 1.0

Julio 2026

Tabla de Contenidos

1. Introducción y propósito
2. Principios de diseño del modelo Firestore
3. Mapa general de colecciones
4. Modelo de usuarios: users
5. Modelo de clientes: clientes
6. Modelo de interacciones comerciales: interacciones
7. Modelo de oportunidades: oportunidades
8. Modelo de propuestas comerciales: propuestas
9. Modelo de auditoría: audit_logs
10. Modelo IA RAG: chatConversaciones y documentosRAG
11. Relaciones lógicas y reglas de propiedad
12. Reglas de seguridad Firestore y acceso backend
13. Validaciones, normalización y consistencia de datos
14. Estados, catálogos y valores canónicos
15. Consultas esperadas e índices recomendados
16. Ciclo de vida de datos, baja lógica y retención
17. Consideraciones para app Flutter y consumidores móviles
18. Riesgos residuales y controles operativos
19. Criterios de aceptación del anexo

1. Introducción y propósito

Este anexo documenta el modelo lógico de datos utilizado por Enci / Enci Ventas sobre Cloud Firestore. Su finalidad es describir las colecciones principales, los campos esperados, las relaciones funcionales, las reglas de propiedad, los estados canónicos, las validaciones aplicadas por backend y los controles mínimos necesarios para pruebas, soporte, UAT y continuidad operativa.

Firestore actúa como persistencia principal del MVP web avanzado. El acceso de negocio no se realiza directamente desde el frontend ni desde consumidores móviles; las operaciones deben pasar por la API FastAPI, que valida token Firebase, usuario activo, rol, plataforma habilitada y propiedad del registro antes de leer o modificar datos.

Criterio central
El modelo Firestore debe entenderse como una base NoSQL controlada por backend. Las reglas de seguridad niegan acceso directo desde cliente; por tanto, la integridad del dato se sostiene mediante API, modelos Pydantic, validaciones de servicio y trazabilidad operativa.

1.1 Responsables del documento

Campo	Detalle
Proyecto	Enci / Enci Ventas
Tipo de documento	Anexo técnico - modelo de datos Firestore
Base técnica	Repositorio oficial Enci (monorepo), backend FastAPI, Firebase Auth y Cloud Firestore
Equipo responsable	Grupo PLM - Universidad Tecnológica Metropolitana
Líder	Joshua Flores
Integrantes	Falon Berrios, Carolaine Palacios y Eliseo Poblete
Versión	1.0
Fecha	Julio 2026

2. Principios de diseño del modelo Firestore

El diseño del modelo prioriza simplicidad operativa, separación por módulos de negocio, trazabilidad mínima y control de acceso por backend. Las colecciones se organizan por entidad principal y se relacionan mediante identificadores de documento, UID de vendedor y referencias funcionales como clienteld y oportunidadld.

- NoSQL documental: cada colección agrupa documentos independientes con campos de negocio normalizados por backend.
- Autorización centralizada: el frontend, la PWA o la app Flutter no determinan permisos; solo consumen la API.

- Propiedad por UID: los registros comerciales asociados a vendedor usan vendedorUid u ownerUid para restringir visibilidad y operación.
- Baja lógica: los clientes se eliminan mediante marca lógica, evitando pérdida irreversible durante operación normal.
- Trazabilidad: cambios críticos y acciones sensibles deben quedar registrables en audit_logs cuando aplica.
- RAG parcial: documentos y conversaciones se modelan como colecciones separadas para historial, fuentes y control de corpus.
- Evolución controlada: nuevos campos o colecciones deben tratarse como cambio técnico documentado para evitar drift de datos.

2.1 Separación entre datos, contrato y presentación

El modelo Firestore no debe copiar la estructura visual exacta de las pantallas. La interfaz puede agrupar o resumir información, pero la persistencia conserva entidades de negocio: usuarios, clientes, interacciones, oportunidades, propuestas, auditoría, conversaciones RAG y documentos RAG.

3. Mapa general de colecciones

Las colecciones principales representan los dominios funcionales del sistema. Cada colección puede crecer con metadatos operativos o campos derivados, siempre que no rompa los contratos documentados de API y validación.

Colección	Dominio	Uso principal	Acceso esperado
users	Identidad y autorización	Usuarios sincronizados desde Firebase Auth con rol, estado y permisos de plataforma.	Backend y endpoints /auth, /me, /users.
clientes	CRM	Clientes comerciales, empresa, contacto, vendedor asignado, estado y baja lógica.	Backend /clientes.
interacciones	Actividad comercial	Registro de llamadas, visitas, correos y reuniones vinculadas a cliente.	Backend /interacciones.
oportunidades	Pipeline	Oportunidades comerciales con etapa, valor estimado y probabilidad.	Backend /oportunidades.
propuestas	Propuestas comerciales	Propuestas vinculadas a oportunidad, montos y estado comercial.	Backend /propuestas.
audit_logs	Auditoría	Eventos técnicos, cambios críticos, cambios temporales de rol y acciones sensibles.	Backend servicios de auditoría.
chatConversaciones	IA RAG	Historial de preguntas, respuestas, fuentes y diagnóstico del asistente.	Backend /rag/conversations y /rag/chat.
documentosRAG	Corpus documental	Metadatos de documentos cargados e indexados para recuperación semántica.	Backend /rag/documents.

3.1 Vista lógica simplificada

Entidad origen	Relación	Entidad destino	Campo de enlace
Firestore Auth	sincroniza	users	uid
users	posee/gestiona	clientes	vendedorUid / ownerUid
clientes	agrupa	interacciones	clienteId
clientes	agrupa	oportunidades	clienteId
oportunidades	contiene	propuestas	oportunidadId
users	registra	audit_logs	usuario.uid / rol
users	genera	chatConversaciones	usuarioid
documentosRAG	alimenta	chatConversaciones	fuentes[].documento

4. Modelo de usuarios: users

La colección users es la fuente operativa de autorización después de validar el ID Token con Firebase Auth. Firebase confirma la identidad; Firestore define si el usuario puede operar, qué rol tiene y qué plataformas puede usar.

Campo	Tipo esperado	Obligatorio	Descripción
uid	string	Sí	Identificador Firebase Auth. Debe coincidir con el document ID.
email	EmailStr/string	Sí	Correo autenticado o de referencia del usuario.
nombre	string	Sí	Nombre normalizado para visualización y operación.
rol	literal	Sí	sin_acceso, admin, supervisor o vendedor.
cargo	string	Sí	Cargo operativo visible en configuración o administración.
rango	literal	Sí	Sin acceso, Solo lectura, Vendedor, Gerente o Administrador.
zona	literal	Sí	Zona norte, Zona centro, Zona sur o Zona oriente.
appMovil	boolean	Sí	Autoriza consumo móvil/PWA/app cuando X-Enci-Client se normaliza como mobile.
webApp	boolean	Sí	Autoriza consumo web cuando X-Enci-Client se normaliza como web.
theme	literal	Sí	Preferencia visual: dark o light.
language	literal	Sí	Preferencia de idioma: es, en o pt.
activo	boolean	Sí	Si es false, bloquea login operativo y endpoints protegidos.
lookupEmail	EmailStr/string	Opcional	Campo de apoyo para búsquedas o administración.
createdAt	datetime	Sí	Fecha de creación del perfil operativo.
updatedAt	datetime	Sí	Última actualización del perfil operativo.

4.1 Documento tipo

```
users/{uid}
{
  "uid": "firebase-uid",
  "email": "usuario@dominio.cl",
  "nombre": "Usuario Enci",
  "rol": "vendedor",
  "cargo": "Vendedor",
  "rango": "Vendedor",
  "zona": "Zona centro",
  "appMovil": true,
  "webApp": true,
  "theme": "dark",
  "language": "es",
  "activo": true,
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

4.2 Reglas operativas

- Un usuario nuevo sincronizado puede quedar con rol sin_acceso y accesos webApp/appMovil deshabilitados hasta habilitación administrativa.
- El campo activo=false debe bloquear el acceso aunque el token Firebase siga siendo válido.
- Los roles admin, supervisor y vendedor se normalizan desde alias conocidos para evitar inconsistencias de escritura.
- Los cambios de rol, estado y accesos deben ejecutarse por endpoints administrativos y no por escritura directa desde cliente.
- Las preferencias propias del usuario se actualizan mediante /users/me/preferences, sin permitir escalamiento de rol ni cambio de estado.

5. Modelo de clientes: clientes

La colección clientes almacena la base CRM. Cada documento representa un cliente o contacto comercial asociado a una empresa, estado de gestión y vendedor responsable. Es la entidad central para interacciones, oportunidades y propuestas.

Campo	Tipo esperado	Obligatorio	Descripción
id	string	Derivado	Identificador del documento retornado por API.
nombre	string	Sí	Nombre del contacto principal. 1 a 120 caracteres.
empresa	string	Sí	Nombre de empresa cliente. 1 a 160 caracteres.
email	EmailStr/string	Sí	Correo de contacto. Debe ser válido.
telefono	string	Opcional	Teléfono normalizado; móvil chileno con 8 dígitos después de +569.
rubro	string	Opcional	Sector o categoría comercial del cliente.
region	string	Opcional	Región o zona geográfica asociada.
estado	literal	Sí	En proceso, Completado o Inactivo.

Campo	Tipo esperado	Obligatorio	Descripción
vendedorUid	string	Sí funcional	UID del vendedor responsable. Usado para visibilidad.
ownerUid	string	Opcional	UID de propietario funcional alternativo o histórico.
createdAt	datetime	Sí	Fecha de creación.
updatedAt	datetime	Sí	Última actualización.
deletedAt	datetime	Opcional	Marca de baja lógica. Si existe, el cliente no debe operar como activo.

5.1 Documento tipo

```

clientes/{clienteId}
{
  "nombre": "María Pérez",
  "empresa": "Agrícola Los Robles",
  "email": "maria.perez@empresa.cl",
  "telefono": "12345678",
  "rubro": "Agroindustrial",
  "region": "Región Metropolitana",
  "estado": "En proceso",
  "vendedorUid": "firebase-uid-vendedor",
  "ownerUid": "firebase-uid-vendedor",
  "createdAt": "timestamp",
  "updatedAt": "timestamp",
  "deletedAt": null
}

```

5.2 Reglas de integridad

- El email debe ser único entre clientes activos cuando la regla funcional esté aplicada por servicio.
- La eliminación se entiende como baja lógica mediante deletedAt; no como eliminación física inmediata del documento.
- El vendedor solo puede operar clientes propios, determinados por vendedorUid u ownerUid.
- Supervisor y admin pueden consultar información agregada según reglas de servicio y rol.
- Campos de texto relevantes rechazan prefijos de fórmula para mitigar formula injection en importaciones o exportaciones.
- El campo vendedorUid no debe ser modificado por vendedores para reasignarse registros ajenos.

6. Modelo de interacciones comerciales: interacciones

La colección interacciones registra la actividad comercial asociada a clientes. Permite construir historial de contacto, seguimiento de reuniones, llamadas, correos y visitas.

Campo	Tipo esperado	Obligatorio	Descripción
id	string	Derivado	Identificador del documento retornado por API.
clienteld	string	Sí	Referencia lógica al documento clientes/{clienteld}.
tipo	literal	Sí	Llamada, visita, correo o reunion.
fecha	datetime	Sí	Fecha de ocurrencia declarada de la interacción.
resumen	string	Sí	Resumen de la interacción. 1 a 1000 caracteres.
resultado	string	Opcional	Resultado o acuerdo alcanzado. Máximo 500 caracteres.
proximaAccion	string	Opcional	Seguimiento comprometido. Máximo 500 caracteres.
vendedorUid	string	Sí	UID del vendedor que registra la interacción.
createdAt	datetime	Sí	Fecha de creación.
updatedAt	datetime	Opcional	Última actualización si el servicio la mantiene.

6.1 Documento tipo

```
interacciones/{interaccionId}
{
  "clienteId": "cliente-id",
  "tipo": "reunion",
  "fecha": "2026-07-06T10:30:00Z",
  "resumen": "Reunión de levantamiento comercial",
  "resultado": "Cliente interesado en propuesta",
  "proximaAccion": "Enviar cotización formal",
  "vendedorUid": "firebase-uid-vendedor",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

6.2 Regla de propiedad

Una interacción debe vincularse a un cliente visible para el usuario que la crea. El vendedor no debe registrar interacciones sobre clientes ajenos. La relación se valida por backend antes de persistir el documento.

7. Modelo de oportunidades: oportunidades

La colección oportunidades representa el pipeline comercial. Cada oportunidad se vincula a un cliente, tiene etapa, valor estimado, probabilidad y descripción. Sirve como base para forecast, embudos y generación de propuestas.

Campo	Tipo esperado	Obligatorio	Descripción
id	string	Derivado	Identificador del documento retornado por API.
clienteld	string	Sí	Referencia lógica a clientes/{clienteld}.
título	string	Sí	Título de la oportunidad. 1 a 160 caracteres.
etapa	literal	Sí	nuevo, contactado, cotizacion, negociacion, ganado o perdido.
valorEstimado	number	Sí	Monto estimado entre 0 y 1.000.000.000.

Campo	Tipo esperado	Obligatorio	Descripción
probabilidad	integer	Sí	Probabilidad entre 0 y 100.
descripcion	string	Opcional	Descripción comercial. Máximo 1000 caracteres.
vendedorUid	string	Sí	UID del vendedor responsable.
createdAt	datetime	Sí	Fecha de creación.
updatedAt	datetime	Sí	Última actualización.

7.1 Documento tipo

```
oportunidades/{oportunidadId}
{
  "clienteId": "cliente-id",
  "titulo": "Renovación de suministro",
  "etapa": "cotizacion",
  "valorEstimado": 2500000,
  "probabilidad": 60,
  "descripcion": "Oportunidad levantada tras reunión técnica",
  "vendedorUid": "firebase-uid-vendedor",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

7.2 Normalización de etapas

Alias recibido	Valor canónico almacenado/operado
prospecto, prospeccion	nuevo
calificado, calificacion	contactado
propuesta, propuesta enviada, cotización	cotizacion
negociación	negociacion
cerrado ganado	ganado
cerrado perdido	perdido

8. Modelo de propuestas comerciales: propuestas

La colección propuestas registra ofertas comerciales vinculadas a una oportunidad y a un cliente. El backend calcula montoDescuento y montoTotal para evitar que cálculos sensibles dependan exclusivamente del cliente visual.

Campo	Tipo esperado	Obligatorio	Descripción
id	string	Derivado	Identificador del documento retornado por API.
clientId	string	Sí	Referencia lógica al cliente.
oportunidadId	string	Sí	Referencia lógica a oportunidades/{oportunidadId}.
titulo	string	Sí	Título de propuesta. 1 a 160 caracteres.
montoNeto	number	Sí	Monto base entre 0 y 1.000.000.000.

Campo	Tipo esperado	Obligatorio	Descripción
descuentoPct	number	Sí	Porcentaje de descuento entre 0 y 100.
montoDescuento	number	Derivado	Monto calculado en backend.
montoTotal	number	Derivado	Monto final calculado en backend.
estado	literal	Sí	borrador, enviada, aceptada o rechazada.
notas	string	Opcional	Notas comerciales. Máximo 1000 caracteres.
vendedorUid	string	Sí	UID del vendedor responsable.
createdAt	datetime	Sí	Fecha de creación.
updatedAt	datetime	Sí	Última actualización.

8.1 Documento tipo

```
propuestas/{propuestaId}
{
  "clienteId": "cliente-id",
  "oportunidadId": "oportunidad-id",
  "titulo": "Propuesta comercial julio",
  "montoNeto": 1000000,
  "descuentoPct": 10,
  "montoDescuento": 100000,
  "montoTotal": 900000,
  "estado": "enviada",
  "notas": "Condiciones sujetas a aprobación",
  "vendedorUid": "firebase-uid-vendedor",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

8.2 Regla de vínculo

Una propuesta debe estar asociada a una oportunidad válida y a un cliente permitido para el usuario. La relación cliente-oportunidad-propuesta evita que se generen documentos comerciales sobre registros ajenos o inconsistentes.

Alias recibido	Valor canónico
ganada	aceptada
perdida	rechazada
en negociación / en negociacion	enviada

9. Modelo de auditoría: audit_logs

La colección audit_logs registra eventos técnicos y cambios críticos que requieren trazabilidad. Su finalidad es facilitar revisión interna, soporte, diagnóstico de incidentes y evidencia de cambios sensibles.

Campo	Tipo esperado	Descripción
usuario	object/string	Identidad del usuario ejecutor o referencia mínima.

Campo	Tipo esperado	Descripción
uid	string	UID Firebase del usuario ejecutor cuando esté disponible.
email	string	Correo del usuario ejecutor cuando esté disponible.
rol	string	Rol efectivo al momento de la acción.
accion	string	Acción registrada: cambio de rol, cambio de estado, importación, evento crítico.
recurso	string	Colección o recurso afectado.
resource_id	string	Identificador del recurso afectado.
metadata	object	Datos complementarios no sensibles.
resultado	string	Resultado esperado si el servicio lo registra.
timestamp	datetime	Fecha/hora del evento.

9.1 Documento tipo

```
audit_logs/{auditLogId}
{
  "uid": "firebase-uid-admin",
  "email": "admin@dominio.cl",
  "rol": "admin",
  "accion": "temporary_role_change",
  "recurso": "users",
  "resource_id": "firebase-uid-afectado",
  "metadata": { "newRole": "supervisor" },
  "timestamp": "timestamp"
}
```

Dato sensible

audit_logs no debe usarse para almacenar tokens, API keys, credenciales, documentos completos ni información personal innecesaria. Debe registrar suficiente contexto para auditoría sin aumentar exposición de datos.

10. Modelo IA RAG: chatConversaciones y documentosRAG

El componente IA RAG se documenta como entrega parcial. Sus colecciones permiten persistir conversaciones por usuario y metadatos de documentos cargados para indexación y recuperación semántica. El índice vectorial Chroma y los archivos documentales pueden residir fuera de Firestore; Firestore conserva metadatos y trazabilidad.

10.1 chatConversaciones

Campo	Tipo esperado	Descripción
id	string	Identificador de conversación retornado por API.
usuariold	string	UID del usuario dueño de la conversación.
usuarioEmail	string	Correo del usuario si está disponible.

Campo	Tipo esperado	Descripción
rol	string	Rol efectivo del usuario.
titulo	string	Título generado o normalizado para identificar la conversación.
mensajes	array<object>	Secuencia de preguntas y respuestas.
createdAt	datetime	Fecha de creación.
updatedAt	datetime	Última actualización.

```

chatConversaciones/{conversacionId}
{
  "usuarioId": "firebase-uid",
  "usuarioEmail": "usuario@dominio.cl",
  "rol": "vendedor",
  "titulo": "Consulta sobre producto",
  "mensajes": [
    {
      "tipo": "pregunta",
      "texto": "consulta del usuario",
      "timestamp": "timestamp"
    },
    {
      "tipo": "respuesta",
      "texto": "respuesta del asistente",
      "timestamp": "timestamp",
      "fuentes": [
        { "documento": "ficha.pdf", "pagina": 3, "fragmento": "texto usado" }
      ],
      "tokens_usados": 512,
      "sin_contexto": false,
      "diagnostico": {
        "origen": "deepseek",
        "proveedor": "DeepSeek",
        "modelo": "deepseek-chat",
        "fragmentos_documentales": 3,
        "fragmentos_internos": 1
      }
    }
  ],
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}

```

10.2 documentosRAG

Campo	Tipo esperado	Descripción
id	string	Identificador del documento.
nombre	string	Nombre sanitizado o nombre operativo del documento.
nombreOriginal	string	Nombre original cargado por el usuario.
descripcion	string	Descripción opcional.
gcsPath	string	Ruta de almacenamiento si se usa Google Cloud Storage.
chunks_count	integer	Cantidad de fragmentos indexados.
subidoPor	string	UID del usuario que carga.
subidoPorEmail	string	Correo del usuario que carga.
tamano_bytes	integer	Tamaño del archivo cargado.

Campo	Tipo esperado	Descripción
activo	boolean	Habilita o deshabilita disponibilidad documental.
createdAt	datetime	Fecha de carga si el servicio la registra.
updatedAt	datetime	Última actualización si aplica.

```
documentosRAG/{documentoId}
{
  "nombre": "ficha-tecnica.pdf",
  "nombreOriginal": "Ficha Técnica Producto.pdf",
  "descripcion": "Documento técnico de producto",
  "gcsPath": "rag/ficha-tecnica.pdf",
  "chunks_count": 42,
  "subidoPor": "firebase-uid-admin",
  "subidoPorEmail": "admin@dominio.cl",
  "tamano_bytes": 1048576,
  "activo": true,
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

10.3 Límite de aceptación del modelo RAG

Las colecciones RAG respaldan el estado actual del componente. No constituyen compromiso de reentrenamiento, tuning, cambio de proveedor, carga masiva de corpus, evaluación avanzada de calidad ni garantía de disponibilidad del proveedor externo.

11. Relaciones lógicas y reglas de propiedad

Firestore no fuerza llaves foráneas como una base relacional. Por ello, las relaciones deben validarse por backend antes de crear o modificar documentos dependientes.

Relación	Regla de validación	Riesgo si no se valida
users.uid -> clientes.vendedorUid	El vendedor solo lista y opera clientes con su UID.	Exposición horizontal de clientes.
clientes.id -> interacciones.clientId	El cliente debe existir y ser visible para el usuario.	Interacciones sobre clientes inexistentes o ajenos.
clientes.id -> oportunidades.clientId	La oportunidad debe crearse solo sobre cliente visible.	Pipeline inconsistente o ajeno.
oportunidades.id -> propuestas.oportunidadId	La propuesta debe vincularse a oportunidad existente y permitida.	Propuestas desalineadas del pipeline.
clientes.id -> propuestas.clientId	El cliente debe coincidir con la oportunidad o estar permitido.	Propuestas cruzadas o ajenas.
users.uid -> chatConversaciones.usuarioId	El vendedor debe ver solo su propio historial; supervisor/admin consolidado si se define.	Exposición de consultas privadas.
documentosRAG.nombre -> fuentes.documento	Las fuentes deben referenciar documentos activos y fragmentos usados.	Respuestas sin trazabilidad documental.

11.1 Propiedad efectiva por rol

Rol	Alcance esperado sobre datos comerciales
vendedor	Clientes, interacciones, oportunidades y propuestas propios. No debe ver ni modificar registros de otros vendedores.
supervisor	Información consolidada y lectura de equipo según endpoints autorizados. No reemplaza al admin en cambios sensibles.
admin	Gestión completa de usuarios, roles, estados, importación CSV y administración general.
sin_acceso	No debe operar datos de negocio aunque exista en Firebase Auth o users.

12. Reglas de seguridad Firestore y acceso backend

El archivo de reglas Firestore mantiene acceso directo denegado desde cliente. Esta decisión reduce el riesgo de escritura directa desde frontend, PWA o app móvil y obliga a que todo flujo pase por backend.

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

12.1 Implicancias

- El frontend React/Vite no debe escribir directamente en Firestore.
- La PWA vendedor o la app Flutter parcialmente realizada deben consumir API FastAPI, no Firestore directo.
- Las credenciales de servicio Firebase Admin deben vivir solo en backend o ambiente seguro.
- La seguridad se implementa en backend mediante `verify_token`, `require_role`, `ensure_platform_access` y validaciones de servicio.
- El cliente visual puede ocultar botones, pero el control real debe ejecutarse siempre en API.

Advertencia

Modificar `firestore.rules` para permitir acceso directo desde cliente cambia el modelo de seguridad del MVP y debe tratarse como cambio técnico mayor, con revisión de reglas, pruebas de abuso y validación formal.

13. Validaciones, normalización y consistencia de datos

La consistencia del modelo depende de validaciones previas al guardado. Estas validaciones se aplican a través de modelos de entrada y servicios de backend, evitando que cada cliente implemente reglas distintas.

Control	Aplicación	Colecciones afectadas
extra=forbid	Rechaza campos no declarados en payloads críticos.	clientes, interacciones, oportunidades, propuestas, users, RAG.
EmailStr	Exige formato de correo válido.	users, clientes.
Formula injection	Rechaza texto que inicia con =, +, - o @.	clientes, interacciones, oportunidades, propuestas.
Normalización teléfono	Convierte teléfono chileno a formato operativo de 8 dígitos después de +569.	clientes.
Normalización roles	Convierte alias a admin, supervisor, vendedor o sin_acceso.	users.
Normalización etapas	Convierte alias comerciales a valores canónicos.	oportunidades.
Normalización estados propuesta	Convierte ganada/perdida a aceptada/rechazada.	propuestas.
Rangos numéricos	Limita montos, descuentos y probabilidades.	oportunidades, propuestas.
Límites de texto	Controla longitudes de nombres, descripciones, resúmenes y notas.	clientes, comercial, RAG.

13.1 Rechazo de fórmula

Prefijos rechazados en campos de texto críticos:

= + - @

Ejemplos rechazados:

=IMPORTXML(...)

+cmd|' /C calc'!A0

@SUMA(A1:A2)

14. Estados, catálogos y valores canónicos

El uso de valores canónicos evita divergencias entre frontend, backend, reportes, importaciones y soporte. Las siguientes listas deben respetarse para crear, actualizar y filtrar datos.

Dominio	Valores canónicos	Uso
roles	sin_acceso, admin, supervisor, vendedor	Control de autorización y administración.
rangos	Sin acceso, Solo lectura, Vendedor, Gerente, Administrador	Visualización/jerarquía operativa.
zonas	Zona norte, Zona centro, Zona sur, Zona oriente	Segmentación comercial.
theme	dark, light	Preferencia visual.
language	es, en, pt	Preferencia de idioma.

Dominio	Valores canónicos	Uso
cliente.estado	En proceso, Completado, Inactivo	Estado CRM.
interaccion.tipo	llamada, visita, correo, reunion	Tipo de contacto.
oportunidad.etapa	nuevo, contactado, cotizacion, negociacion, ganado, perdido	Pipeline comercial.
propuesta.estado	borrador, enviada, aceptada, rechazada	Estado de propuesta.
RAG mensaje.tipo	pregunta, respuesta	Historial conversacional.
RAG diagnostico.origen	deepseek, local, other	Trazabilidad de respuesta.

15. Consultas esperadas e índices recomendados

Firestore puede requerir índices compuestos para consultas con filtros múltiples, ordenamientos o combinaciones por vendedor, estado, etapa y fechas. La necesidad exacta se confirma al ejecutar los flujos en ambiente UAT con datos reales.

Colección	Consulta esperada	Índice recomendado / criterio
clientes	vendedorUid + estado + búsqueda/limit	Índice por vendedorUid y estado si se combinan filtros frecuentes.
clientes	deletedAt ausente + email	Soporte a validación funcional de email único entre activos desde servicio.
interacciones	vendedorUid + clienteld + fecha	Índice para historial por cliente y vendedor.
oportunidades	vendedorUid + etapa + clienteld	Índice para pipeline y detalle de cliente.
propuestas	vendedorUid + estado + oportunidadId	Índice para listado comercial y seguimiento.
users	activo + limit	Índice simple o consulta directa según volumen.
chatConversaciones	usuariold + updatedAt desc	Índice para historial privado del vendedor.
documentosRAG	activo + nombre	Índice o consulta simple según cantidad de documentos.

15.1 Criterio de performance

- Para MVP y UAT, los límites de listado reducen riesgo de lectura masiva accidental.
- Si Firestore solicita un índice compuesto durante pruebas, el enlace de creación debe registrarse como evidencia técnica y aplicarse en el ambiente correspondiente.
- Las consultas de dashboard deben revisarse con volumen real antes de salida productiva de alto uso.
- Las consultas RAG a Firestore no reemplazan el índice vectorial; solo complementan contexto interno e historial.

16. Ciclo de vida de datos, baja lógica y retención

El ciclo de vida de datos define cómo se crean, modifican, desactivan o excluyen registros de operación. En MVP, se prioriza conservar trazabilidad y evitar pérdida irreversible de datos comerciales.

Entidad	Creación	Actualización	Baja / cierre
users	Login/register sincroniza desde Firebase Auth.	Admin modifica rol, estado y accesos; usuario modifica preferencias.	activo=false bloquea operación.
clientes	Vendedor o admin crea vía API; CSV solo admin.	PATCH con reglas de propiedad y rol.	deletedAt marca baja lógica.
interacciones	Vendedor registra sobre cliente visible.	Actualización si el servicio la expone o se define.	Conservación como historial comercial.
oportunidades	Vendedor crea sobre cliente visible.	Cambio de etapa, monto, probabilidad y descripción.	etapa ganado/perdido cierra comercialmente.
propuestas	Vendedor crea vinculada a oportunidad.	Cambio de estado y montos según permisos.	estado aceptada/rechazada cierra seguimiento.
chatConversaciones	Se crea al primer mensaje si no hay conversacion_id.	Se agregan turnos y se actualiza título/updatedAt.	Retención según política futura.
documentosRAG	Admin carga e indexa documento.	Reindexación o activación/desactivación.	activo=false o eliminación controlada si se define.

16.1 Datos que no deben persistirse

- ID Tokens Firebase o refresh tokens.
- API keys de DeepSeek, Firebase Admin o credenciales de servicio.
- Contraseñas o secretos de proveedores externos.
- Archivos completos en audit_logs o conversaciones, salvo metadatos estrictamente necesarios.
- Datos personales no requeridos por operación comercial, soporte o trazabilidad.

17. Consideraciones para app Flutter y consumidores móviles

La aplicación móvil Flutter se considera entregable parcial externo al monorepo, conforme al contexto de cierre acordado. Desde la perspectiva de datos, su interacción correcta con Firestore debe realizarse por API FastAPI y no por escritura directa contra Firestore.

Aspecto	Criterio
Autenticación	La app obtiene token Firebase y lo envía como Authorization: Bearer <token>.
Plataforma	Debe enviar X-Enci-Client: mobile o equivalente para validar appMovil.
Datos	Debe consumir endpoints existentes de clientes, oportunidades, propuestas, dashboard o RAG según alcance parcial.
Restricción	No se garantiza cobertura completa de endpoints desde Flutter dentro del alcance parcial.
Seguridad	No debe incluir service account ni credenciales backend dentro del APK.

Aspecto	Criterio
Continuidad	Nuevas pantallas, publicación en tiendas, sincronización offline avanzada o integración total quedan como Fase 2 o nuevo acuerdo.

Interpretación de cierre

La ausencia de la app Flutter en el repositorio oficial no invalida su existencia como entregable parcial externo. Para el modelo de datos, el criterio relevante es que cualquier integración futura respete la API, los roles, el campo appMovil y las reglas de propiedad ya documentadas.

18. Riesgos residuales y controles operativos

El modelo documentado cubre el cierre de Development y preparación UAT. Existen riesgos residuales normales para una salida productiva controlada, especialmente si cambian reglas comerciales, volumen de datos, integraciones externas o accesos móviles.

Riesgo	Impacto	Control recomendado
Campos nuevos sin documentación	Inconsistencia entre API, frontend y reportes.	Todo campo nuevo debe registrarse en modelo y contrato.
Acceso directo a Firestore desde cliente	Bypass de reglas backend.	Mantener deny-all y revisar credenciales.
Falta de índices compuestos	Errores o latencia en consultas UAT.	Crear índices solicitados por Firestore y registrar evidencia.
Datos duplicados por importación	CRM contaminado o reportes erróneos.	Validación de duplicados y revisión CSV previa.
Desalineación de etapas/estados	Pipeline o dashboards incorrectos.	Usar valores canónicos y normalización backend.
RAG sin corpus suficiente	Respuestas sin contexto o incompletas.	Documentar corpus cargado y límites de aceptación parcial.
Flutter con integración incompleta	Consumo parcial o inconsistente de API.	Declarar alcance parcial y pruebas funcionales específicas.

19. Criterios de aceptación del anexo

Este anexo se considera aceptable si permite comprender la estructura de persistencia, validar la relación entre colecciones y API, ejecutar pruebas UAT y sostener soporte técnico sin depender de conocimiento informal del equipo.

Código	Criterio	Resultado esperado
CA-D01	Se identifican las colecciones principales de Firestore.	El cliente y soporte comprenden el alcance de persistencia.
CA-D02	Se describen campos esperados de users, clientes, interacciones, oportunidades y propuestas.	Los datos comerciales quedan trazables y verificables.
CA-D03	Se documentan audit_logs, chatConversaciones y documentosRAG.	La auditoría y RAG parcial tienen modelo claro.

Código	Criterio	Resultado esperado
CA-D04	Se explican relaciones lógicas y reglas de propiedad.	UAT puede validar restricciones de vendedor, supervisor y admin.
CA-D05	Se incorpora la regla Firestore deny-all.	Queda claro que la autorización reside en backend.
CA-D06	Se listan valores canónicos, validaciones y normalizaciones.	Se reducen ambigüedades de estados y payloads.
CA-D07	Se distinguen riesgos residuales de fallas de implementación.	La salida productiva queda condicionada a ambiente, datos reales e índices necesarios.
CA-D08	Se considera Flutter como consumidor móvil parcial externo.	No se confunde ausencia en GitHub con inexistencia del entregable parcial.

19.1 Checklist mínimo de recepción

- Verificar que firestore.rules mantenga deny-all para acceso directo desde cliente.
- Crear usuarios de prueba con roles vendedor, supervisor, admin, sin_acceso e inactivo.
- Crear clientes propios y ajenos para probar restricciones de propiedad.
- Registrar interacciones, oportunidades y propuestas vinculadas al mismo cliente.
- Validar cálculo server-side de montos de propuesta.
- Ejecutar importación CSV de clientes con casos válidos, duplicados y campos maliciosos.
- Probar una conversación RAG con fuente documental y una sin contexto suficiente.
- Registrar índices requeridos por Firestore durante UAT si aparecen errores de consulta.

Cierre del anexo

El modelo de datos Firestore documentado respalda el MVP web avanzado, la operación comercial, la administración de usuarios, la trazabilidad técnica y el componente IA RAG parcial. Toda ampliación estructural debe quedar documentada como cambio técnico o fase posterior.